Arquitecturas de redes de computadores
Tema I

Protocolos de transporte



Rafael Sebastian/ Juan M. Orduña
Departamento de Informática
Escuela Técnica Superior de Ingenierías
Universitat de València
Adaptado de Rogelio Montañana



Objetivos sección

- Entender el funcionamiento de los mecanismos de TCP
- Diferenciar entre TCP y UDP
- Asimilar el funcionamiento de protocolos de ventana deslizante



Protocolo TCP/IP

- **Protocolo UDP**
- Protocolo TCP

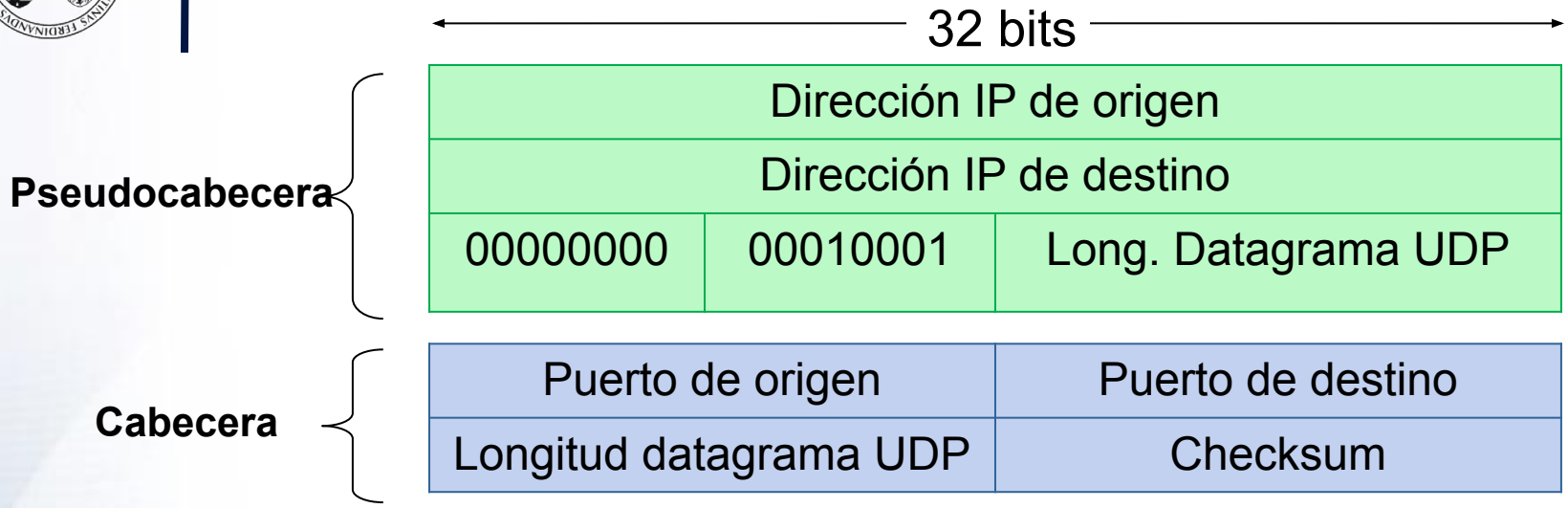
Funciones generales del nivel de transporte

- Se encarga del transporte de los datos extremo a extremo (host a host)
- Realiza la comunicación de forma transparente al medio físico. Usa los servicios del nivel de red
- Multiplexa tráfico de diversas instancias (procesos) del nivel de aplicación. El nivel de transporte (como el de red) tiene una sola instancia en el host
- El servicio que ofrece puede ser de dos tipos:
  - **Orientado a conexión:** garantiza la entrega de los datos, sin pérdidas ni duplicados. Ej.: TCP (Internet), TP4 (OSI)
  - **No orientado a conexión**: equivale al servicio que ofrece IP, pero a nivel de transporte. Ej.: UDP (Internet), TP0 (OSI)

Protocolo UDP

- Servicio sencillo, CLNS, no fiable
- Se utiliza en los siguientes entornos:
  - El intercambio de mensajes es muy escaso, ej.: consultas al DNS (servidor de nombres)
  - La aplicación es en tiempo real y no puede esperar confirmaciones. Ej.: videoconferencia, voz sobre IP
  - Los mensajes se producen regularmente y no importa si se pierde alguno. Ej: NTP, SNMP
  - El medio de transmisión es altamente fiable y sin congestión (LANs). Ej: NFS
  - Se envía tráfico broadcast/multicast

La cabecera UDP



La pseudo-cabecera se añade al principio del datagrama para el cálculo del *checksum*, pero no se envía. Permite a UDP comprobar que IP no se ha equivocado (ni le ha engañado) en la entrega del datagrama.

El valor $10001_2 = 17_{10}$ indica que el protocolo de transporte es UDP

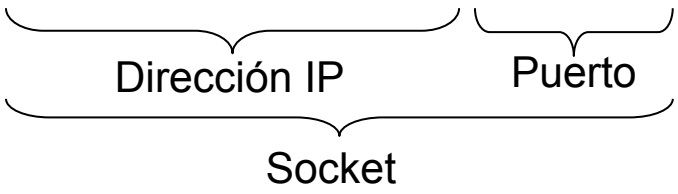


Protocolo UDP

- Las TPDUs de UDP se denominan **mensajes** o **datagramas UDP**
- UDP multiplexa los datos de las aplicaciones y efectúa opcionalmente una comprobación de errores, pero **NO** realiza:
  - Control de flujo
  - Control de congestión
  - Retransmisión de datos perdidos
  - Conexión/desconexión

Multiplexación

- La multiplexación se realiza mediante el puerto (origen o destino) que puede valer de 0 a 65535.
- Los puertos 0 a 1023 están reservados para servidores 'bien conocidos' ('well known ports')
- La combinación de una dirección IP y un puerto identifica un 'socket' (origen o destino de los datagramas UDP): 147.156.135.22.1038



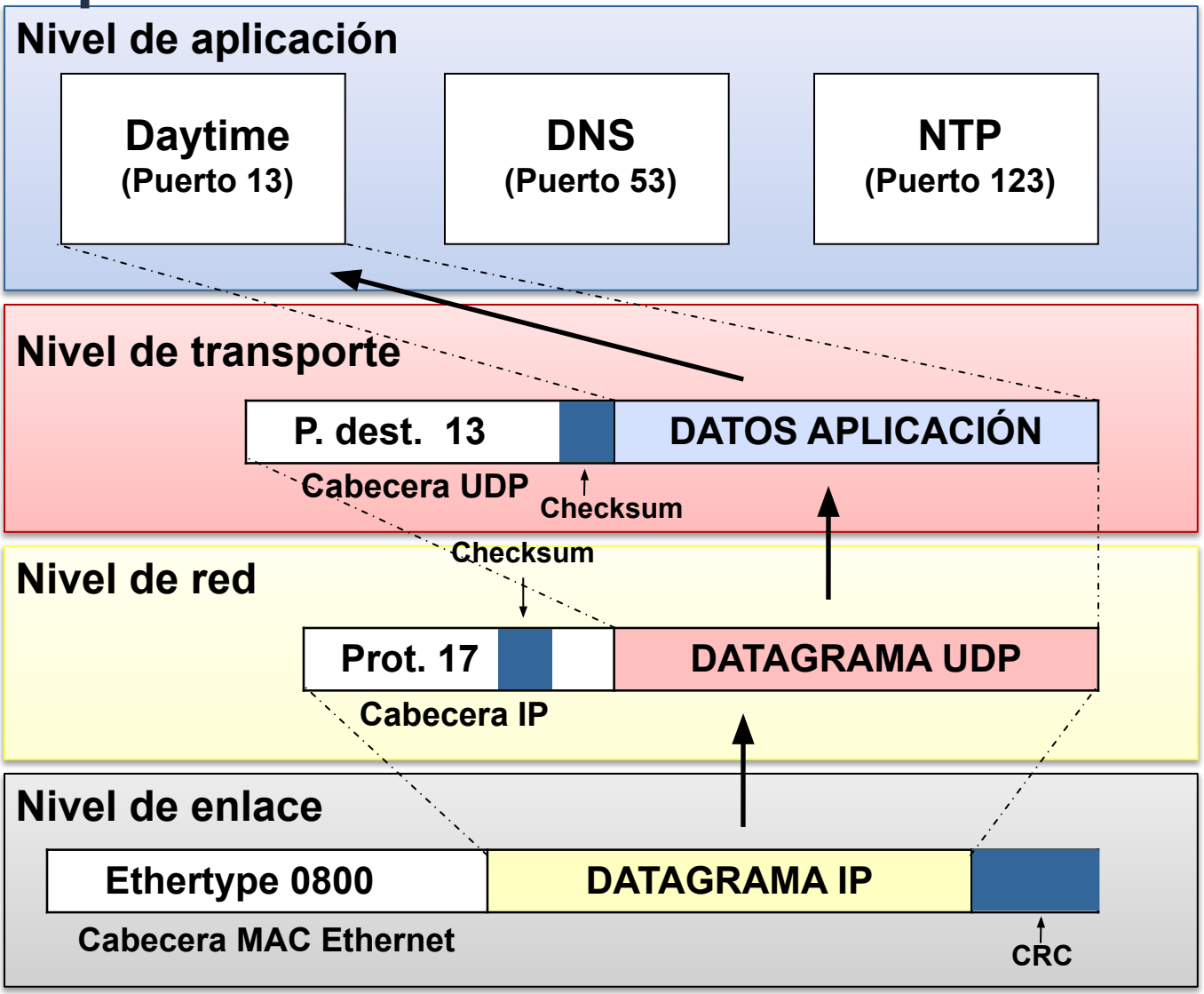
Multiplexación

Múltiples instancias (una o varias por protocolo)

Dos instancias (TCP y UDP)

Una instancia IP (puede haber otros protocolos)

Múltiples instancias (una por interfaz)





Multiplexación: API de Sockets

Socket: Una puerta entre el proceso de aplicación y el protocolo de transporte (UCP o TCP).

- Un interfaz local al host creado por la aplicación.
- Controlado por el S.O.
- Permite a los procesos de aplicación enviar y recibir mensajes hacia/desde otros procesos de aplicación multiplexando instancias de la capa de transporte.
- Centrado en paradigma cliente/servidor



Sockets UDP

UDP: no hay “conexión” entre cliente y servidor

- El emisor debe indicar explícitamente la dirección IP y el puerto del destino para cada paquete.

- El servidor debe extraer la dirección IP y el puerto del emisor del paquete.

UDP: los datos pueden recibirse desordenados o incluso perderse.

Ejemplo socket UDP en C (1)

» Cliente

» Servidor

```
struct sockaddr_in dirsock, emisor;
int sockservidor, ret, frlen=sizeof(emisor);
char      *buf[2000];

sockservidor=socket(PF_INET,SOCK_DGRAM,0);
if (sockservidor==-1) ERROR();
dirsock.sin_family=AF_INET;


dirsock.sin_addr.s_addr=INADDR_ANY;
dirsock.sin_port=htons(53);
ret= bind(sockservidor, (struct
    sockaddr*)&dirsock, sizeof(dirsock));
if (ret==-1) ERROR();
```

Crear el socket UDP

Asignar puerto...

Ejemplo socket UDP en C (2)

» Cliente

Crear el socket UDP...

```
int sockcliente, ret;
struct sockaddr_in dirsock;
struct hostent *resolvhost;

sockcliente=socket(PF_INET,SOCK_DGRAM,0);
if (sockcliente==-1) ERROR();

dirsock.sin_family=AF_INET;
resolvhost=gethostbyname("servidor.tlm.unavarra.es");
if (resolvhost==NULL) ERROR();
dirsock.sin_addr.s_addr=*(u_long*)resolvhost-
    >h_addr_list[0];
dirsock.sin_port=htons(53);
```

Enviar datagrama...

```
ret=sendto(sockcliente, buf, max, 0, (struct
    sockaddr*)dirsock,  sizeof(dirsock));
```

» Servidor

Esperar a recibir

```
ret=recvfrom(sockservidor, buf,
    2000, 0, (struct
    sockaddr*)&emisor, &frlen);
```

```
if (ret==-1) ERROR();
```

Ejemplo cliente UDP en Java



```java
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetSocketAddress;
import java.net.SocketTimeoutException;


public class udp_cliente_daytime {
  public static void main(String args[]) throws java.io.IOException {
    // Figure out the host and port we're going to talk to
    String host = args[0];
    int port = 13;
    if (args.length > 1)
      port = Integer.parseInt(args[1]);

    DatagramSocket socket = new DatagramSocket();

    socket.setSoTimeout(3000);

    byte[] buffer = new byte[512];
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length, new InetSocketAddress(host,
       port));

    for (int i = 0; i < 3; i++) {
      try {
        packet.setLength(0);
        socket.send(packet);

        packet.setLength(buffer.length);
        socket.receive(packet);

        System.out.print(new String(buffer, 0, packet.getLength(), "US-ASCII"));
        break;
      } catch (SocketTimeoutException e) {
        System.out.println("No response before the timeout");
      }
    }

    socket.close();
  }
}
```

Ejemplo servidor UDP en Java

```
import java.net.*;
import java.io.*;

public class ServidorUDP {

 public static void main (String args[]) {

  try {

   DatagramSocket socketUDP = new DatagramSocket(6789);
   byte[] bufer = new byte[1000];

   while (true) {
    // Construimos el DatagramPacket para recibir peticiones
    DatagramPacket peticion =
     new DatagramPacket(bufer, bufer.length);

    // Leemos una petición del DatagramSocket
    socketUDP.receive(peticion);

    System.out.print("Datagrama recibido del host: " +
                     peticion.getAddress());
    System.out.println(" desde el puerto remoto: " +
                     peticion.getPort());

    // Construimos el DatagramPacket para enviar la respuesta
    DatagramPacket respuesta =
     new DatagramPacket(peticion.getData(), peticion.getLength(),
                        peticion.getAddress(), peticion.getPort());

    // Enviamos la respuesta, que es un eco
    socketUDP.send(respuesta);
   }

  } catch (SocketException e) {
   System.out.println("Socket: " + e.getMessage());
  } catch (IOException e) {
   System.out.println("IO: " + e.getMessage());
  }
 }

}
```



Protocolo TCP/IP

- Protocolo UDP
- **Protocolo TCP**

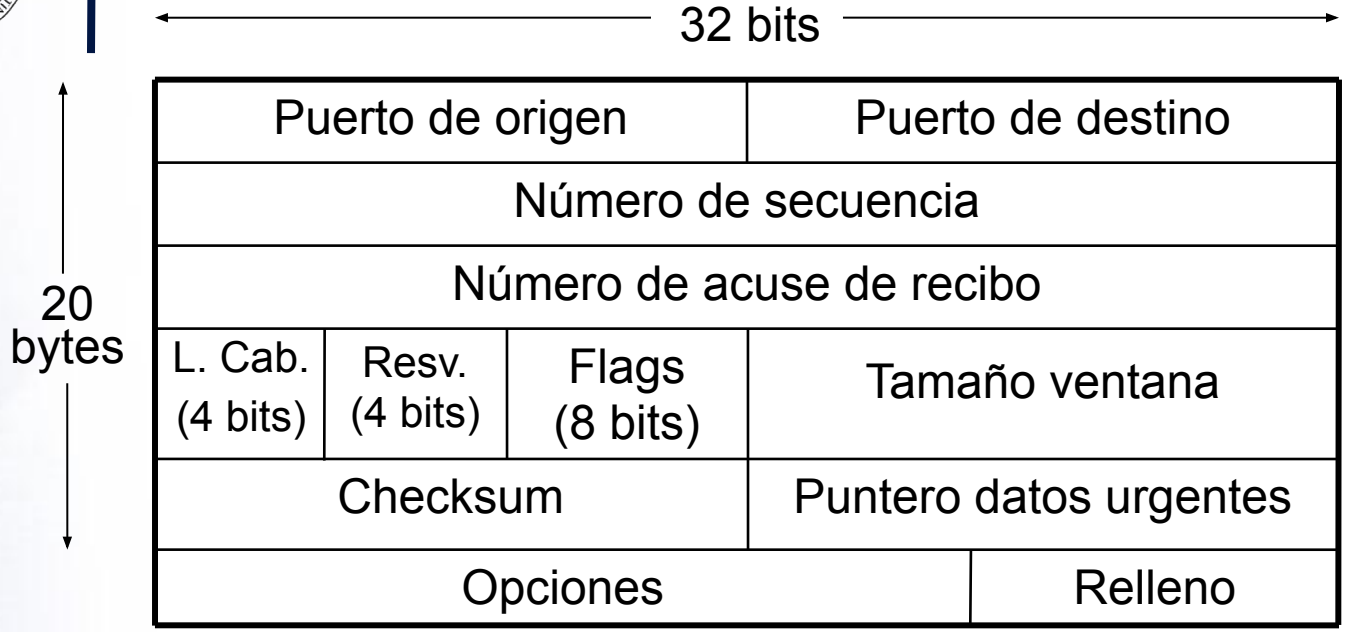
TCP
Transmission Control Protocol

- El protocolo TCP ofrece el servicio de transporte orientado a conexión (CONS) en Internet
- Está diseñado para ofrecer un transporte fiable sobre un servicio no fiable del nivel de red (el que le suministra IP)
- Las TPDUs de TCP se llaman **segmentos**
- El TCP actual se especificó en el RFC 793 en 1981 y sigue plenamente vigente

Funciones de TCP

- Multiplexar el nivel de aplicación (port)
- Establece y terminar conexiones
- Controlar errores, retransmitiendo segmentos perdidos o erróneos. Eliminar duplicados
- Gestionar los buffers y ejercer control de flujo de forma eficiente
- Gestionar el intercambio de datos con las aplicaciones
- Efectuar control de congestión de la red

La cabecera TCP



Flags: CWR: Congestion Window Reduced
ECE: ECN Echo (ECN=Explicit Congestion Notification)
URG: el segmento contiene datos urgentes
ACK: el campo número de acuse de recibo tiene sentido
PSH: el segmento contiene datos 'Pushed'
RST: ha habido algún error y la conexión debe cerrarse
SYN: indica el inicio de una conexión
FIN: indica el final de una conexión

Multiplexación

- Se utiliza el número de puerto (origen o destino) como en UDP. Puede valer de 0 a 65535
- Los puertos 0 a 1023 están reservados para servidores ‘bien conocidos’
- La combinación de dirección IP y puerto identifica el ‘socket’
- Una conexión TCP queda especificada por los dos sockets que se comunican (IP origen-puerto origen, IP destino-puerto destino)



Algunos servicios 'bien conocidos'

Servicio	Puerto	TCP	UDP
DayTime	13	X	X
FTP	**21**	X	
SSH	**22**	X	
TelNet	23	X	
SMTP	**25**	X	
Domain (DNS)	53	X	X
BOOTP	67		X
TFTP	69		X
HTTP	**80**	X	
POP3	110	X	
NTP	123		X
SNMP	161		X
LDAP	389	X	

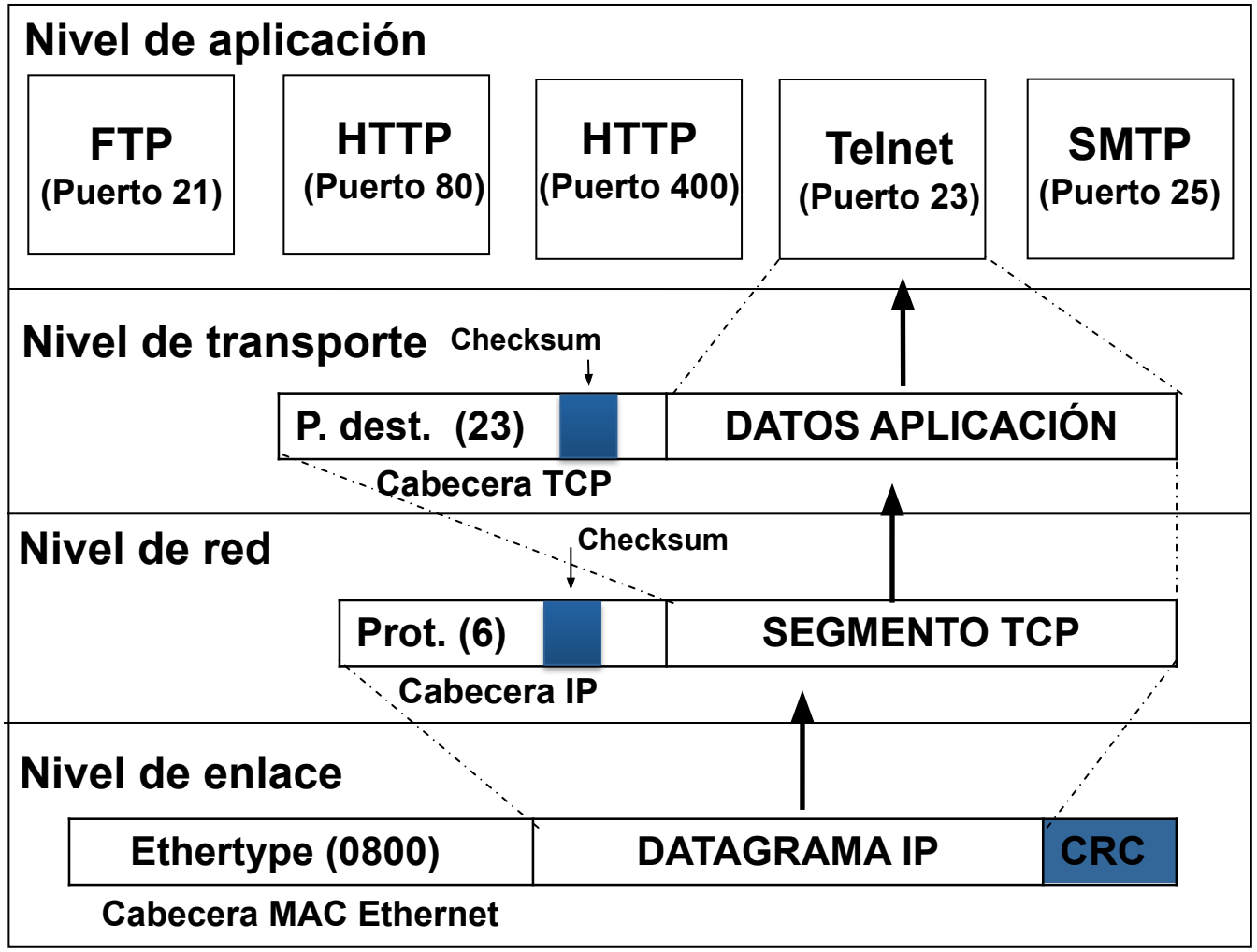
Multiplexación

Múltiples instancias (una o varias por protocolo)

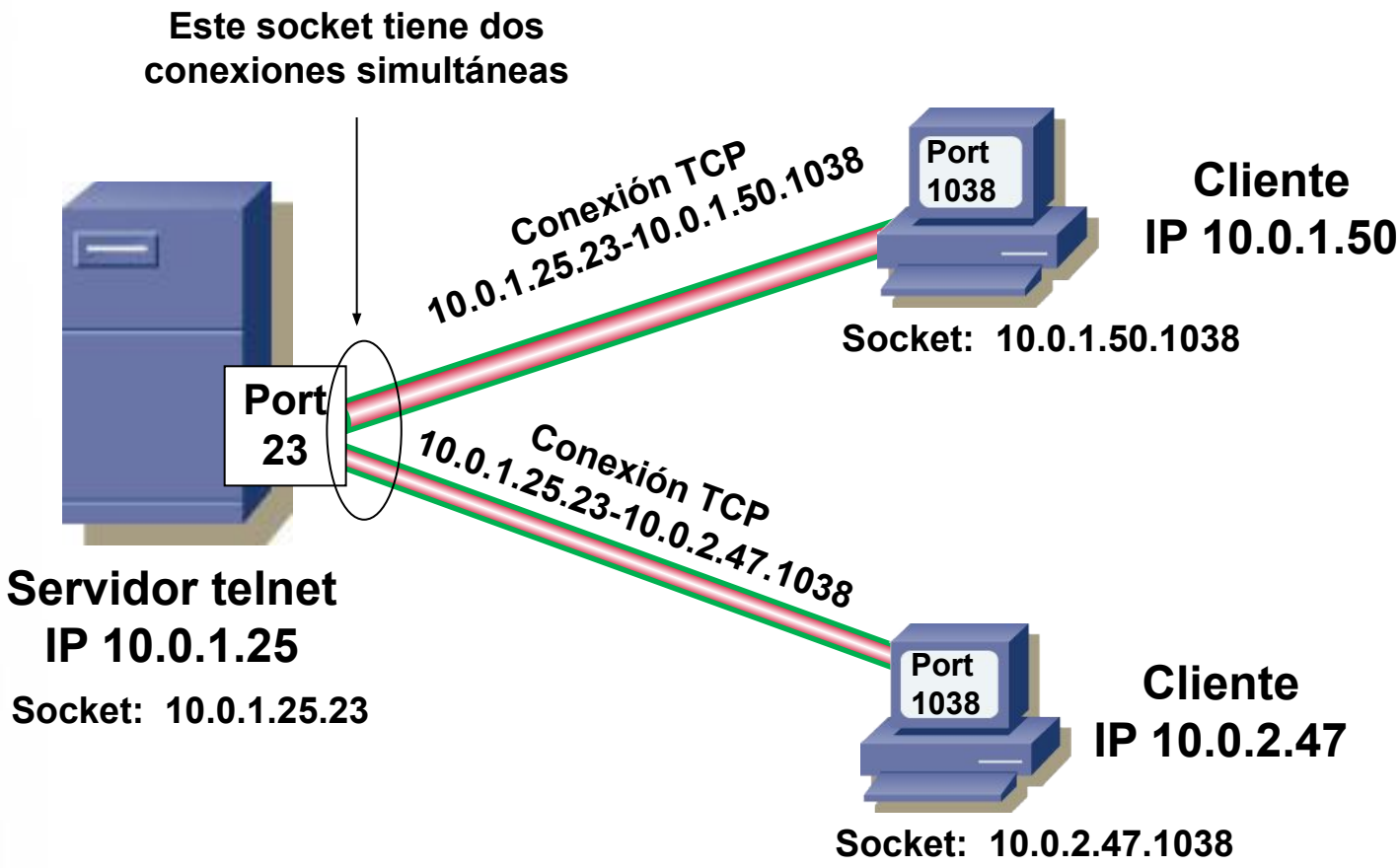
Dos instancias (TCP y UDP)

Una instancia IP (puede haber otros protocolos)

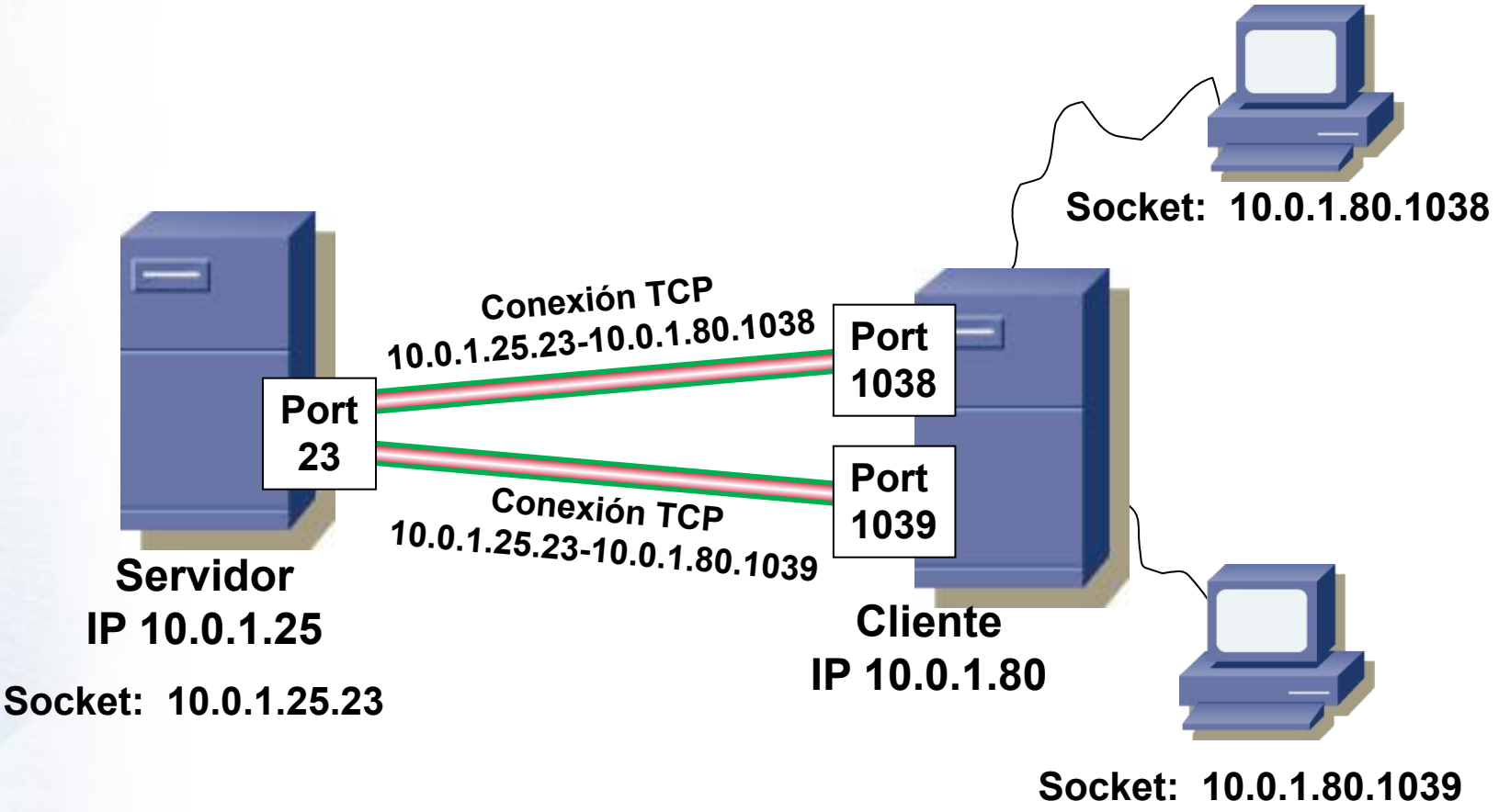
Múltiples instancias (una por interfaz)



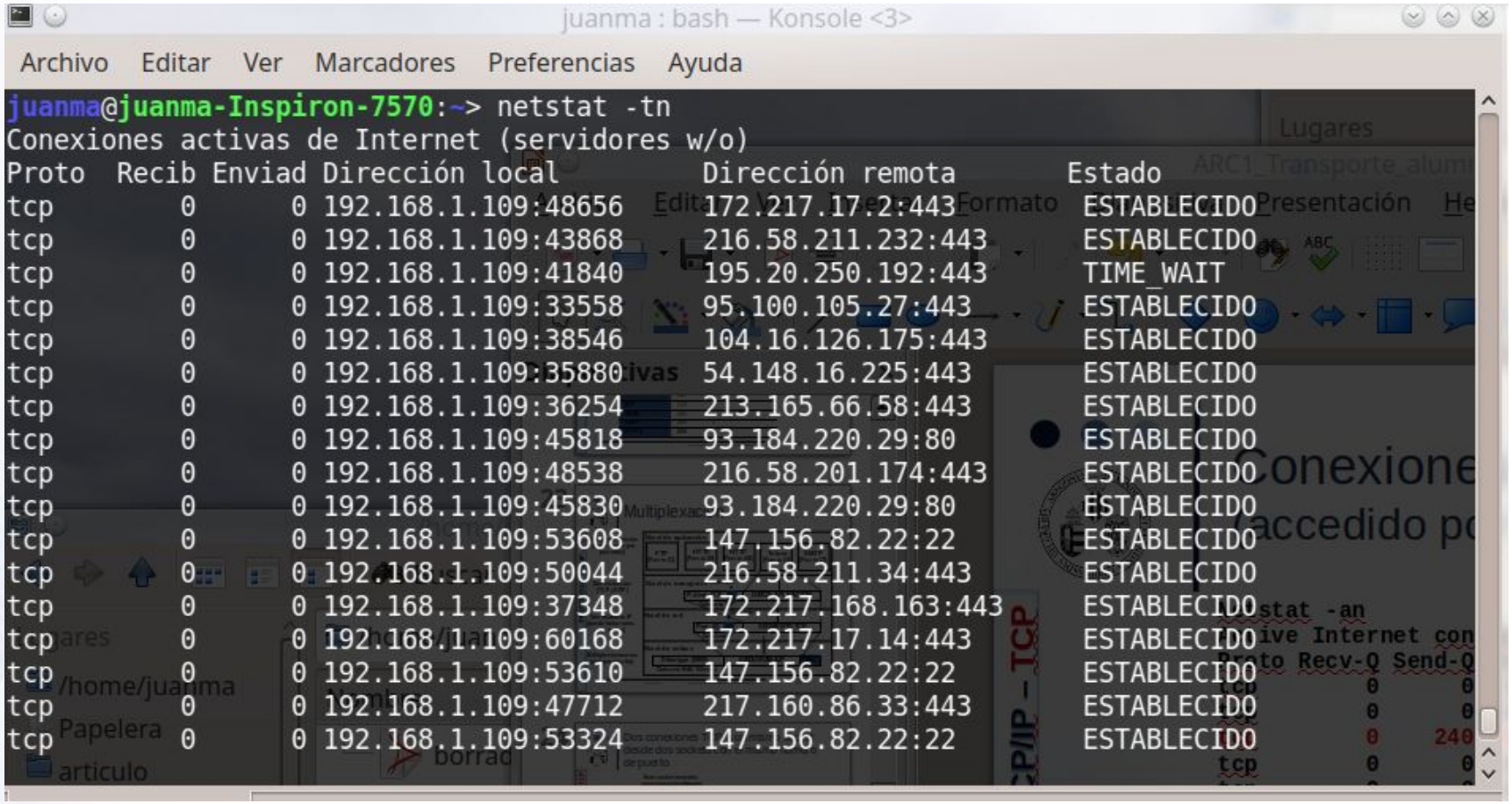
Dos conexiones TCP a un mismo socket desde dos sockets con el mismo número de puerto



Dos conexiones TCP a un mismo socket desde dos sockets con la misma dirección IP



Ejemplo conexiones TCP en host



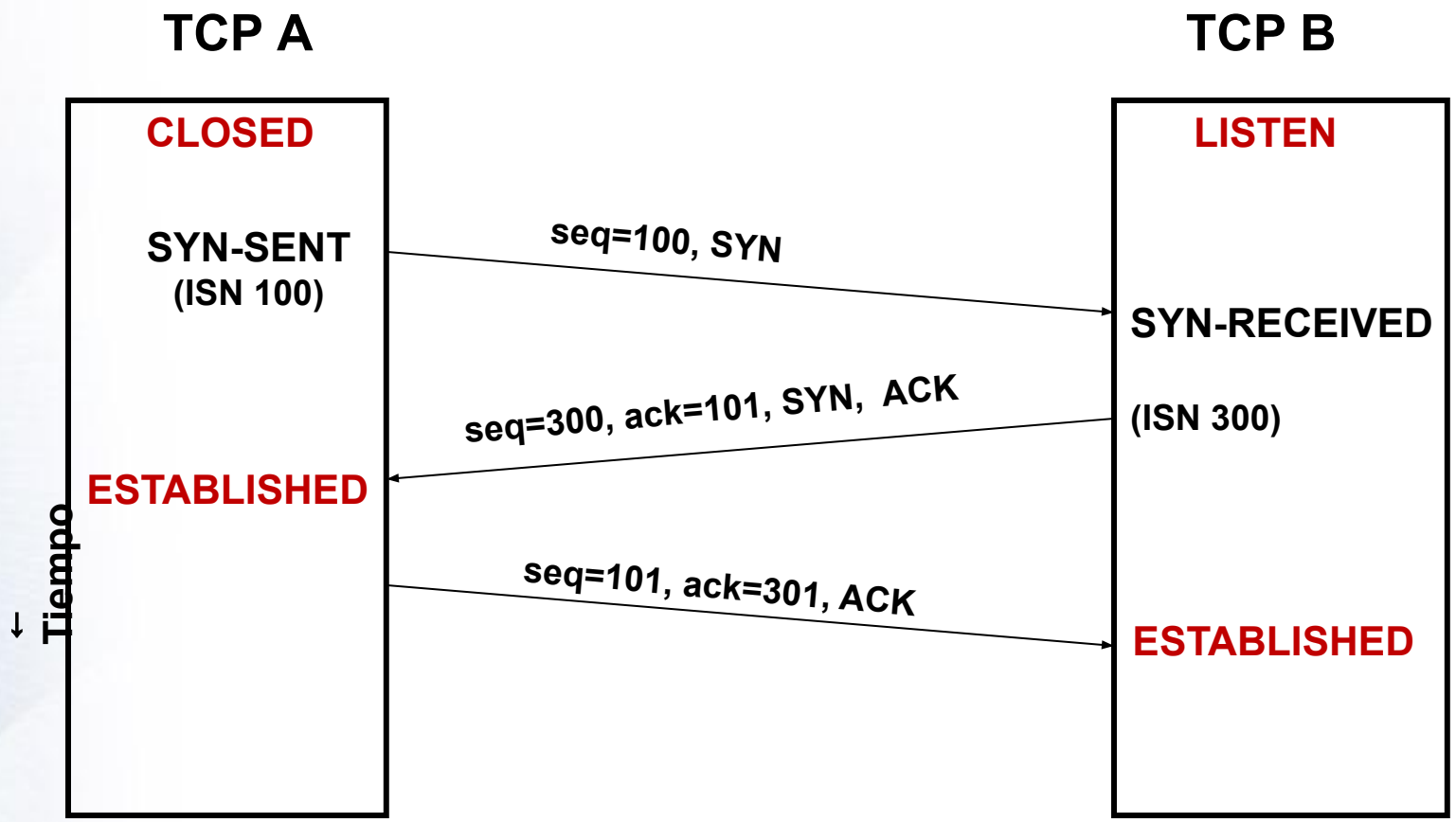
Conexión por 'Saludo a tres vías'

- Los segmentos pueden llegar duplicados (p. ej. se pierde la confirmación de un segmento con lo que el emisor lo reenvía)
- Con un procedimiento de conexión simple los segmentos duplicados podrían causar problemas. Una sesión entera podría duplicarse.
- Para evitar los problemas debidos a duplicados se utiliza un procedimiento de conexión más elaborado denominado saludo a tres vías
- El saludo a tres vías se basa en la elección de un número que identifica de forma única cada intento de conexión y que actúa como PIN. De este modo se evita el riesgo de aceptar como válidos segmentos retrasados que pudieran aparecer fruto de conexiones anteriores

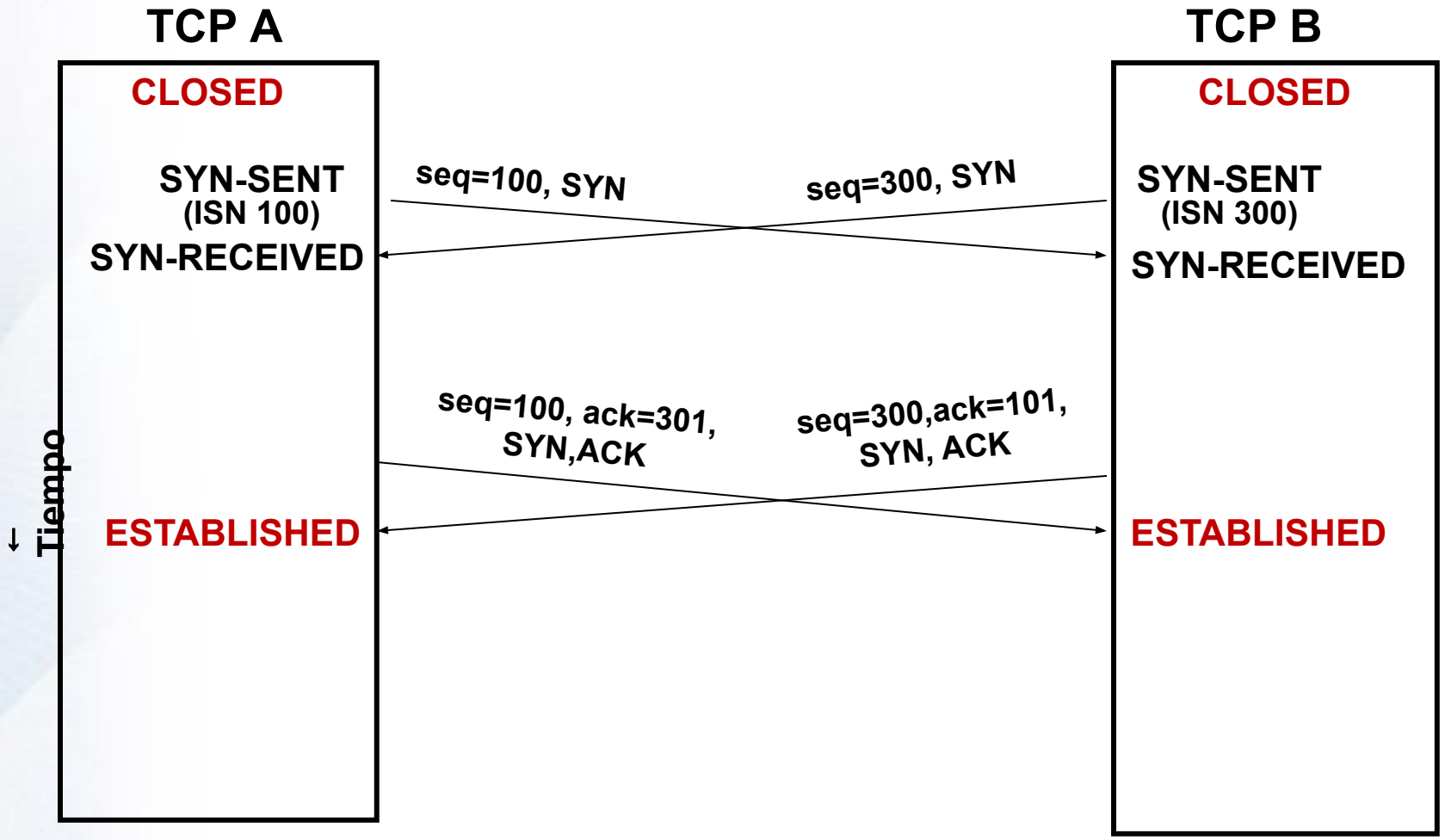
Procedimiento del saludo a tres vías

1. El cliente elige para cada intento de conexión un número único. El número elegido lo incluye en la petición de conexión que envía al servidor
2. El servidor, cuando recibe la petición, elige otro número único y envía una respuesta al cliente indicándoselo
3. El cliente al recibir la respuesta considera establecida la conexión. A continuación envía un tercer mensaje en el que acusa recibo del anterior. El servidor considera establecida la conexión cuando el recibe este tercer mensaje

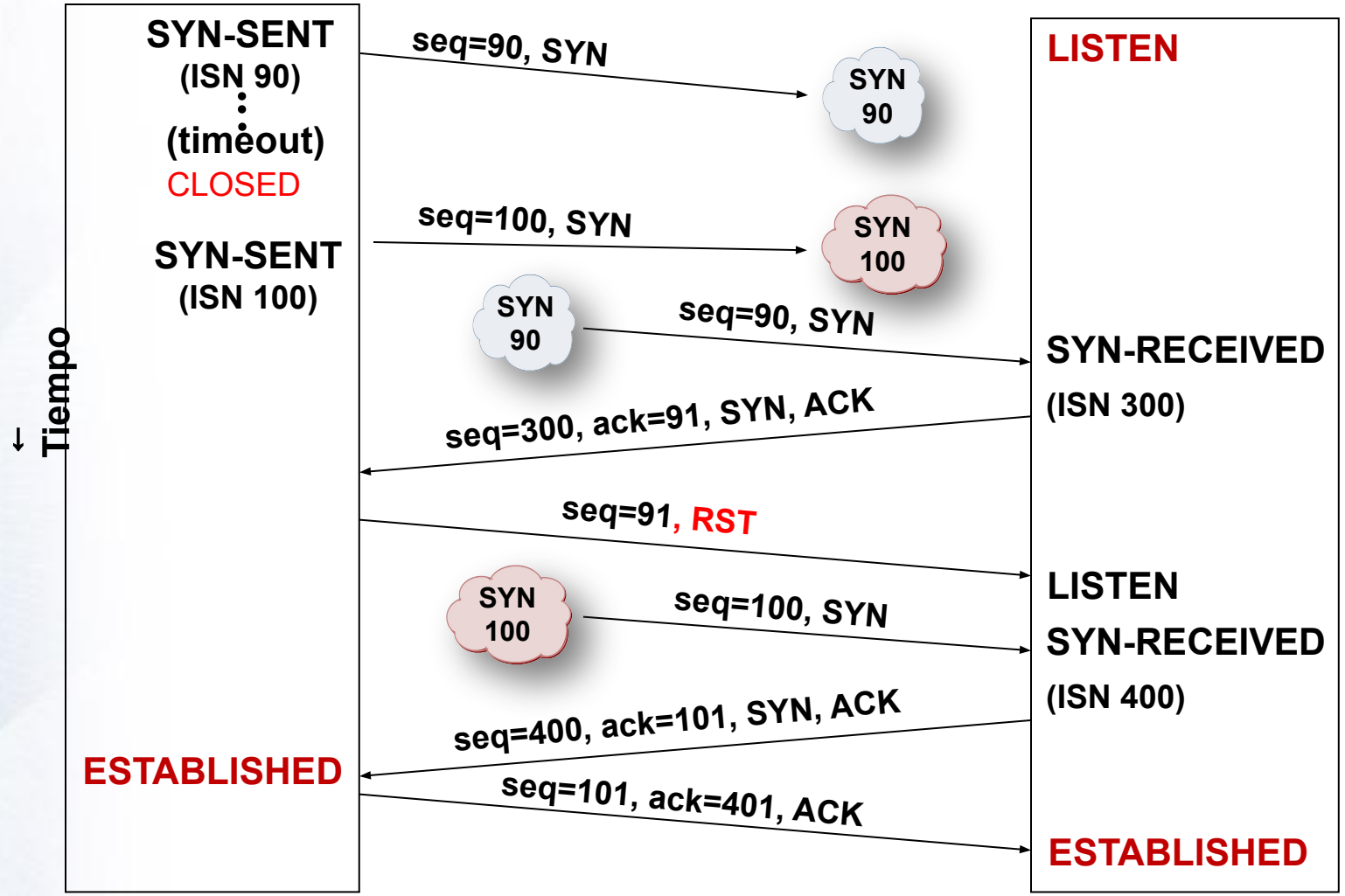
Establecimiento de una conexión TCP por saludo a tres vías



Saludo a tres vías, conexión simultánea



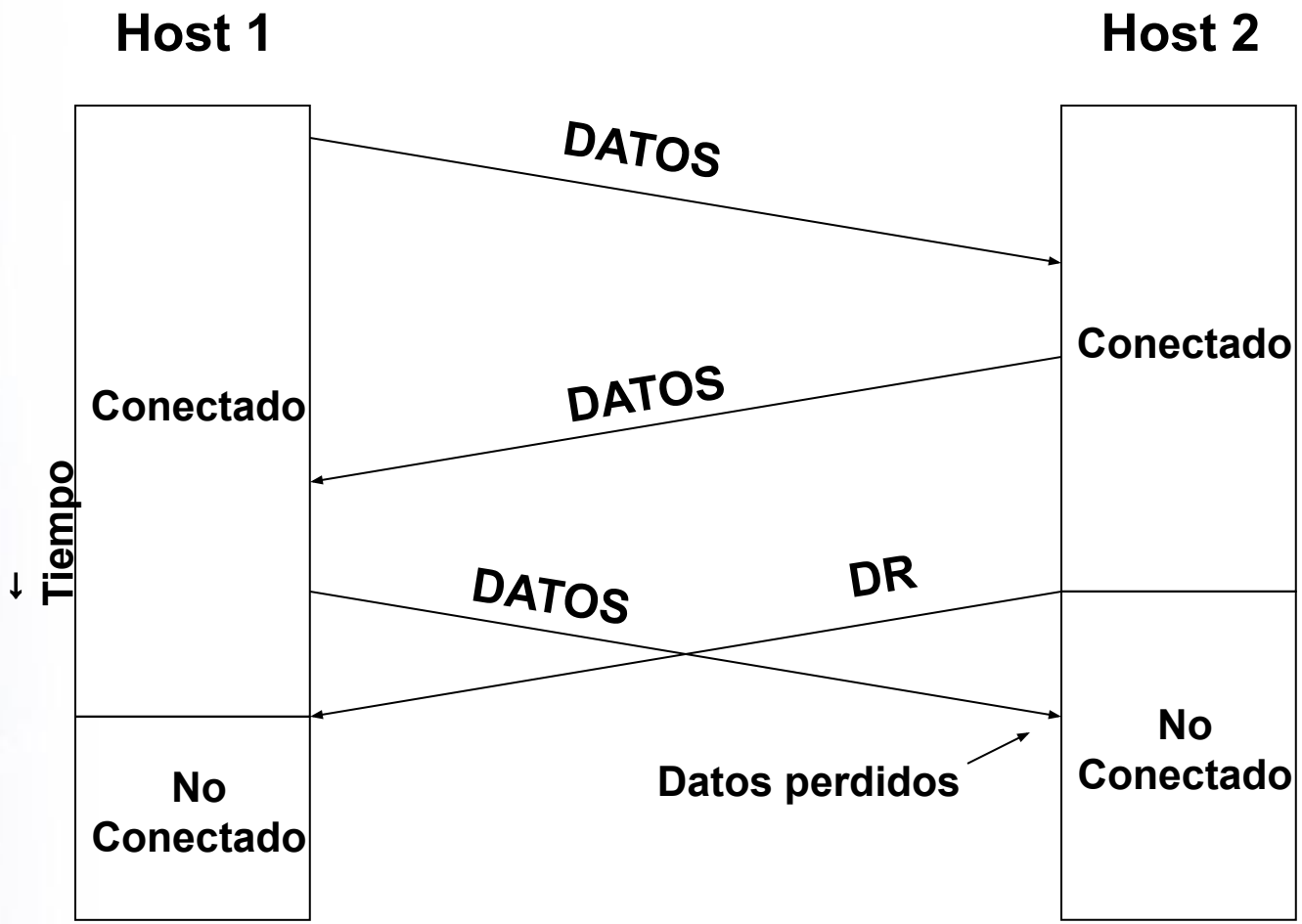
Conexión con SYN duplicado



Desconexión

- Puede ser de dos tipos:
  - *Simétrica*: la conexión se considera formada por dos circuitos simplex y cada host solo puede cerrar uno (aquel en el que él emite datos). El cierre de un sentido se interpreta como una 'invitación' a cerrar el otro
  - *Asimétrica:* desconexión unilateral (un host la termina en ambos sentidos sin esperar a recibir confirmación del otro). Puede provocar pérdida de información

Desconexión asimétrica

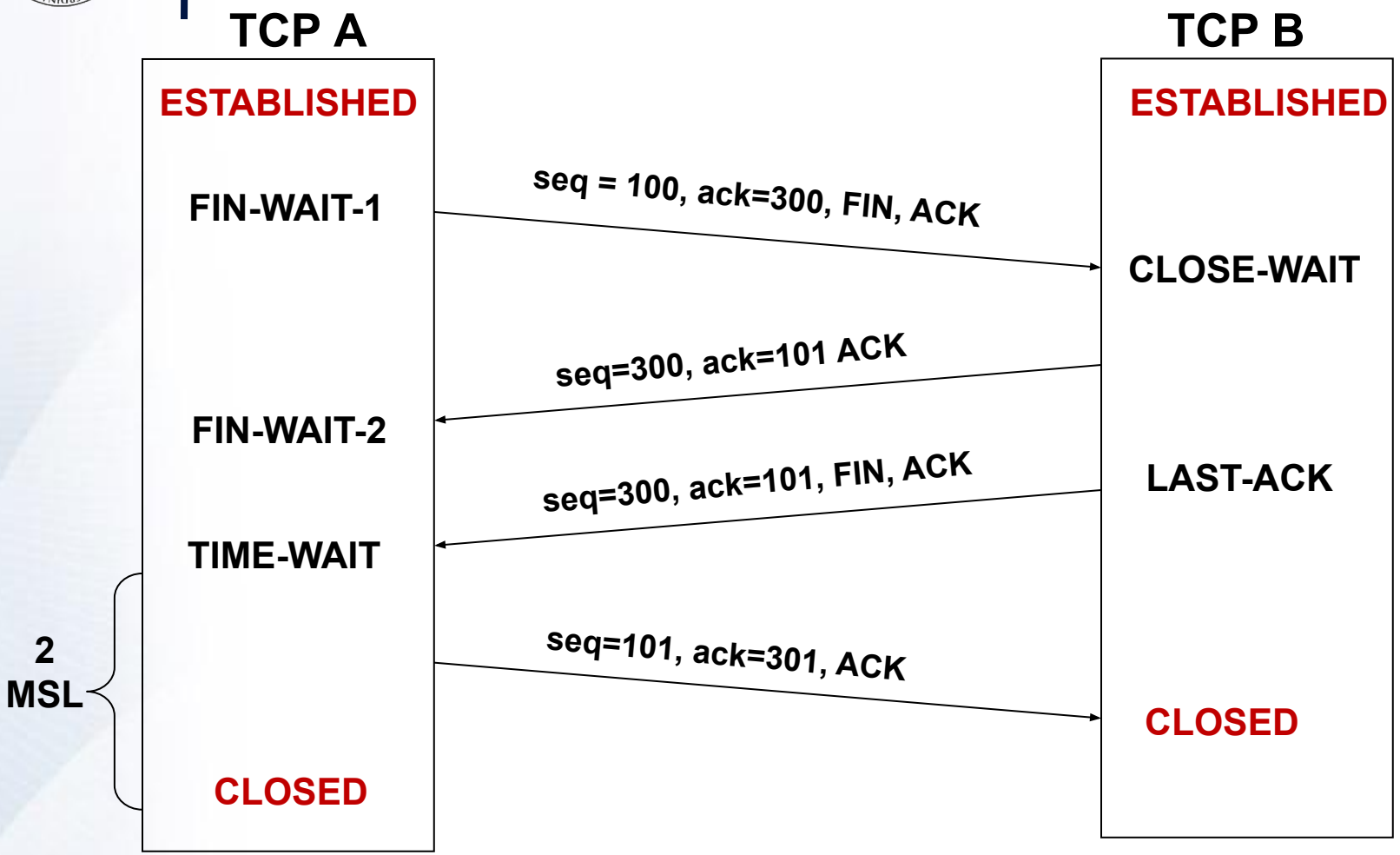


DR: Disconnect Request

Desconexión por saludo a tres vías

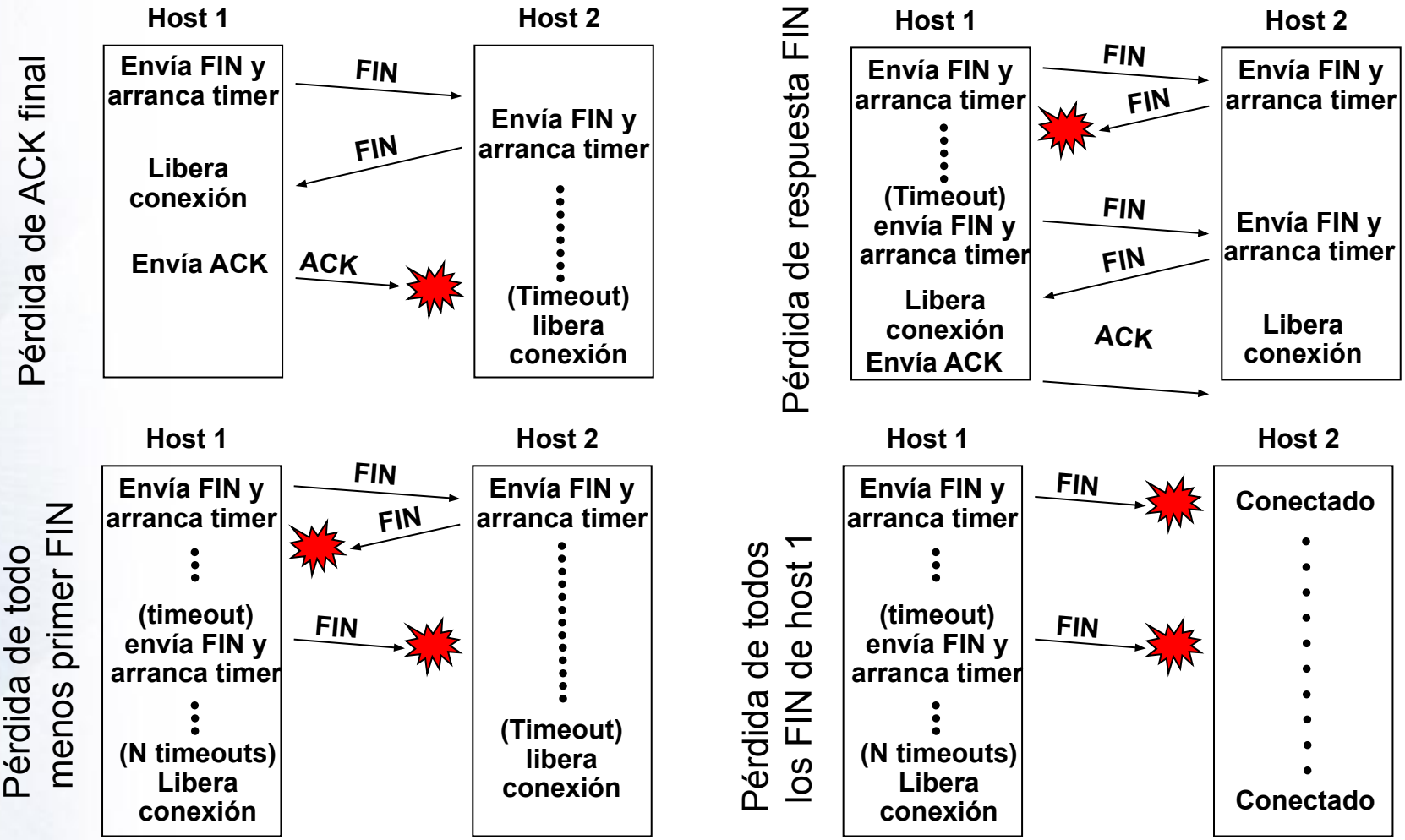
- Se trata de una desconexión simétrica en la que se tiene una seguridad razonable de que no se pierden datos
- Supone el intercambio de tres mensajes, de forma análoga a la conexión, de ahí su nombre
- En caso de que alguno de los mensajes de desconexión se pierda una vez iniciado el proceso la conexión se termina por *timeout*

Desconexión a tres vías



MSL: Maximum Segment Lifetime (normalmente 2 minutos)

Desconexión a tres vías, casos anormales



Números de secuencia y flags

- El número de secuencia es el que corresponde al primer byte enviado en ese segmento
- TCP incrementa el número de secuencia de cada segmento según los bytes que tenía el segmento anterior, con una sola excepción:

  Los flags SYN y FIN, cuando están puestos, incrementan en 1 el número de secuencia
- Esto permite que se pueda acusar recibo de un segmento SYN o FIN sin ambigüedad
- Podemos considerar que los segmentos que tienen puesto el flag SYN o FIN llevan un byte de datos 'virtual'
- La presencia del flag ACK no incrementa el número de secuencia



Intercambio de datos
TCP ↔ aplicación

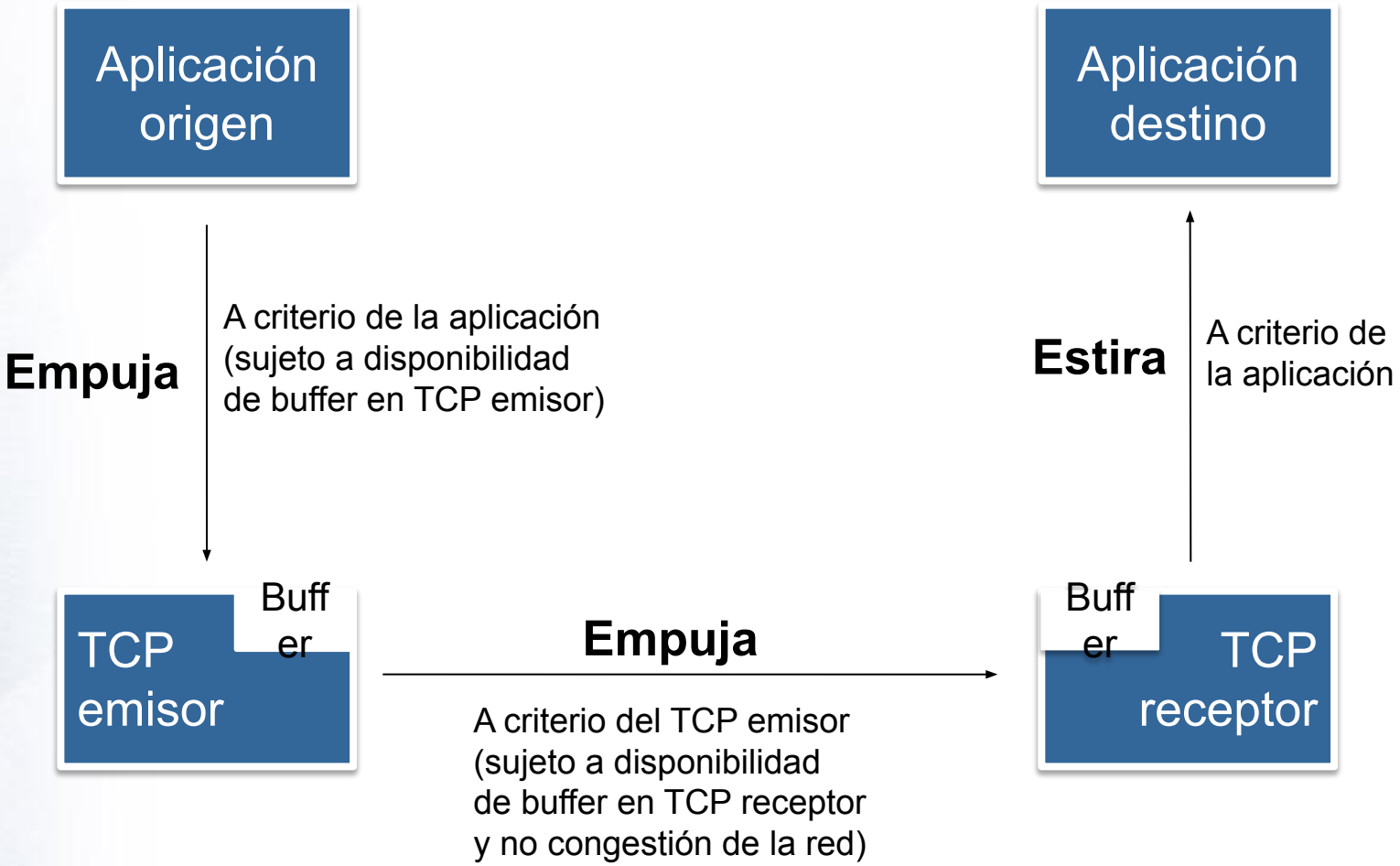
- **Aplicación → TCP**: la aplicación envía los datos a TCP cuando quiere (siempre y cuando TCP tenga espacio libre en el buffer de emisión)
- **TCP → Aplicación**: la aplicación lee del buffer de recepción de TCP cuando quiere y cuanto quiere. Excepción: datos urgentes
- Para TCP los datos de la aplicación son un flujo continuo de bytes, independientemente de la separación que pueda tener la aplicación (registros, etc.). Es responsabilidad de la aplicación asegurarse que esa separación (si existe) se mantenga después de transmitir los datos

Intercambio de datos
TCP ↔ TCP

- El TCP emisor manda los datos cuando quiere
- El TCP emisor decide el tamaño de segmento según sus preferencias. Al inicio de la conexión se negocia el MSS (Maximum Segment Size)
- Cada segmento ha de viajar en un datagrama IP
- Normalmente TCP intenta agrupar los datos para que los segmentos tengan la longitud máxima, reduciendo así el overhead debido a cabeceras y proceso de segmentos
- El TCP emisor puede aplicar la técnica de descubrimiento de la MTU del trayecto ('Path MTU Discovery', MTU = Maximum Transfer Unit) para ajustar el MSS al tamaño óptimo para esa comunicación

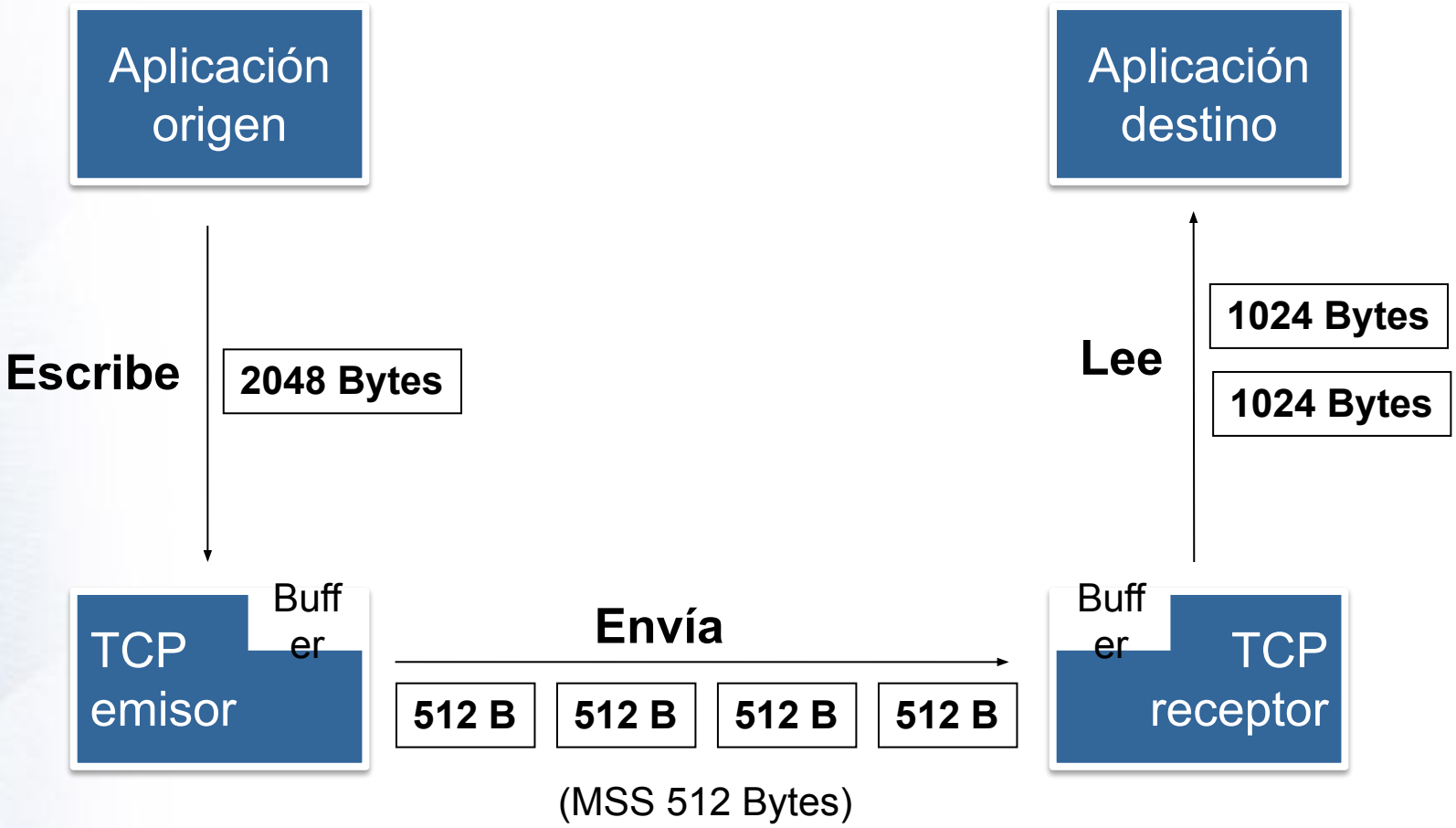
Intercambio de datos

TCP ↔ Aplicación
TCP ↔ TCP

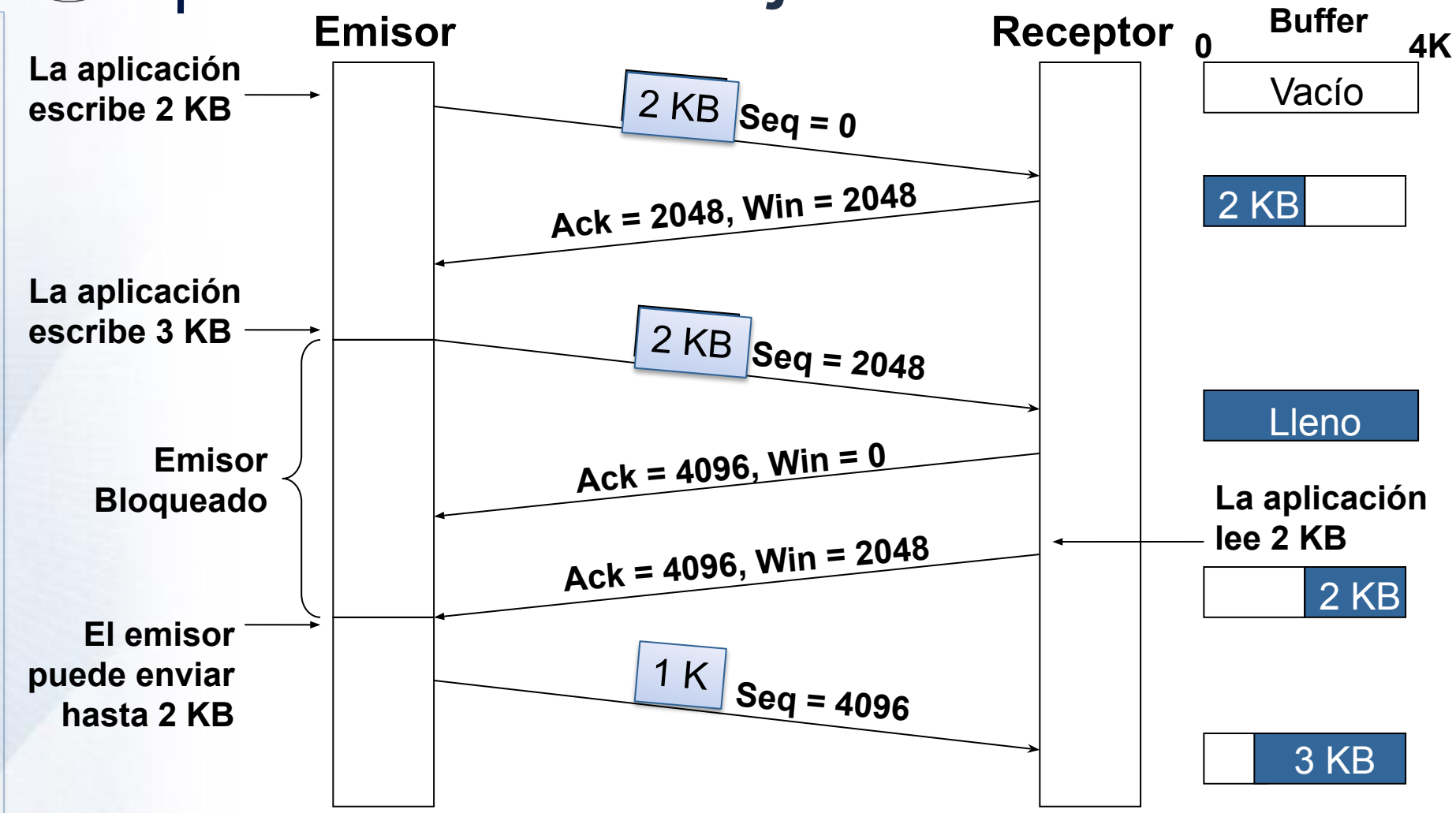


Intercambio de datos

TCP ↔ Aplicación
TCP ↔ TCP



Gestión de buffers y Control de flujo



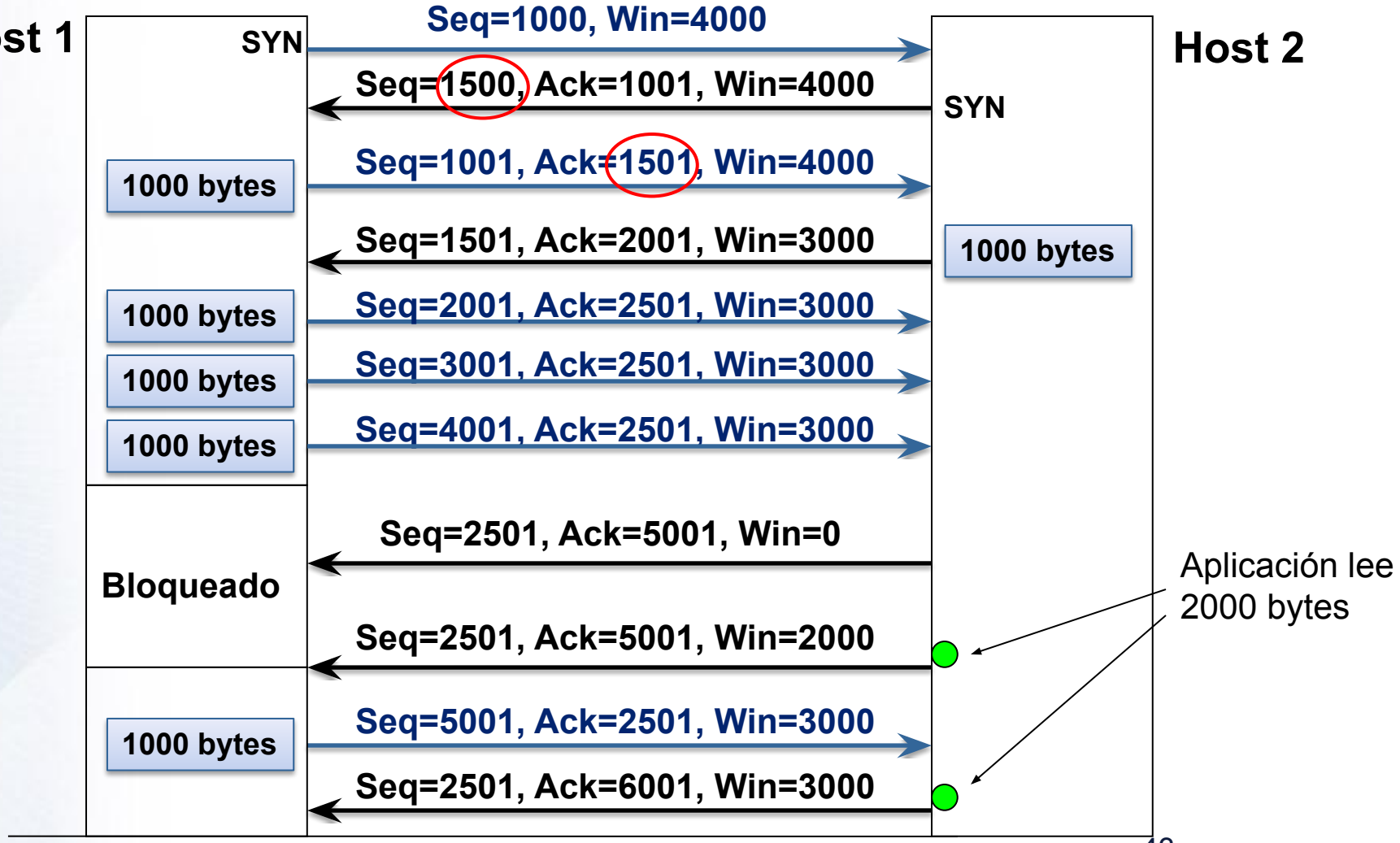
Gestión de buffers y control de flujo

- El TCP receptor nunca debería retirar el espacio en buffer que ya ha anunciado al emisor
- Sin embargo TCP debe estar preparado por si el del otro lado lo hace (esto se denomina 'contraer la ventana')

(Recordemos la Ley de Postel):

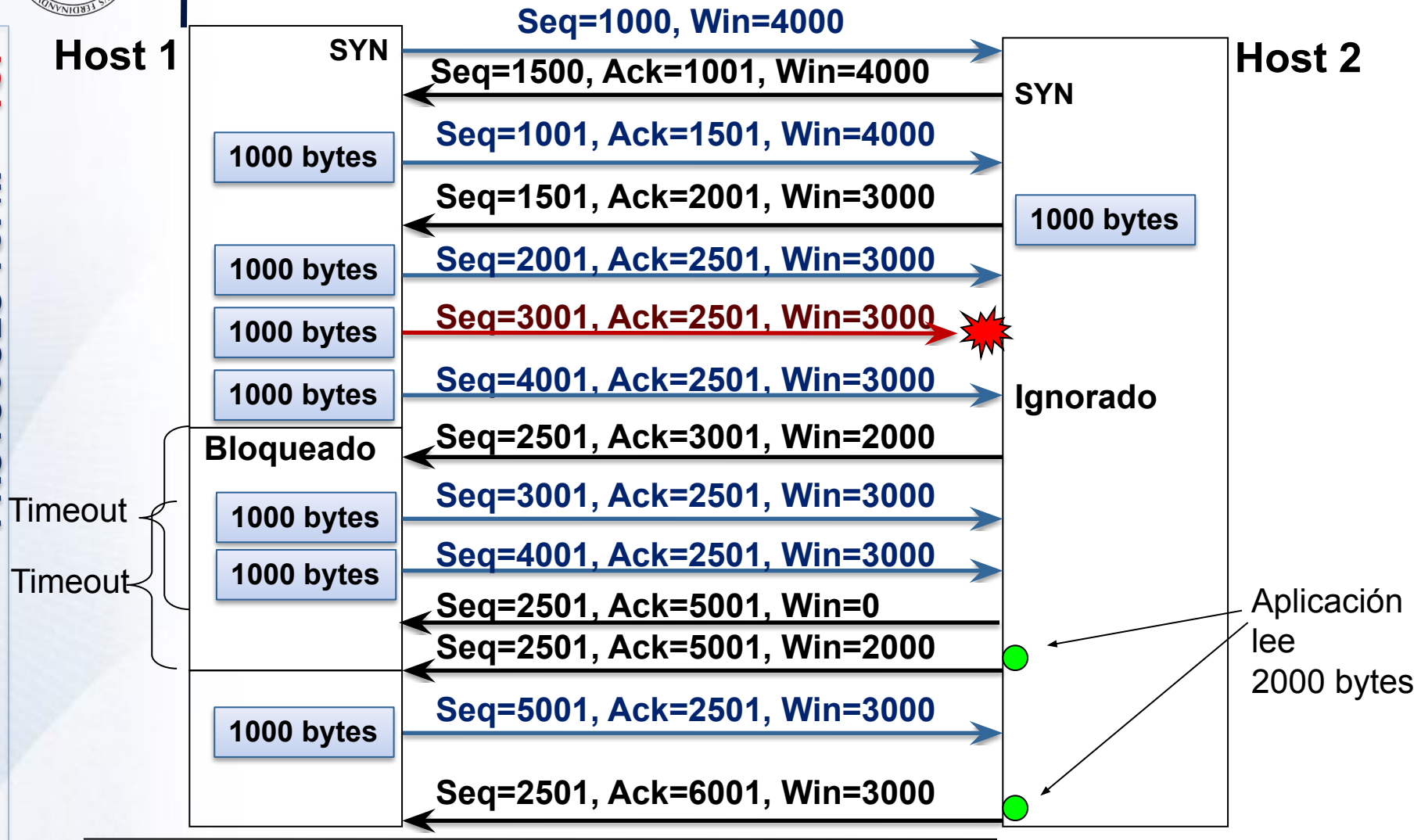
'Sé estricto al enviar y tolerante al recibir'

Control de flujo y números de secuencia. Caso normal, sin pérdidas



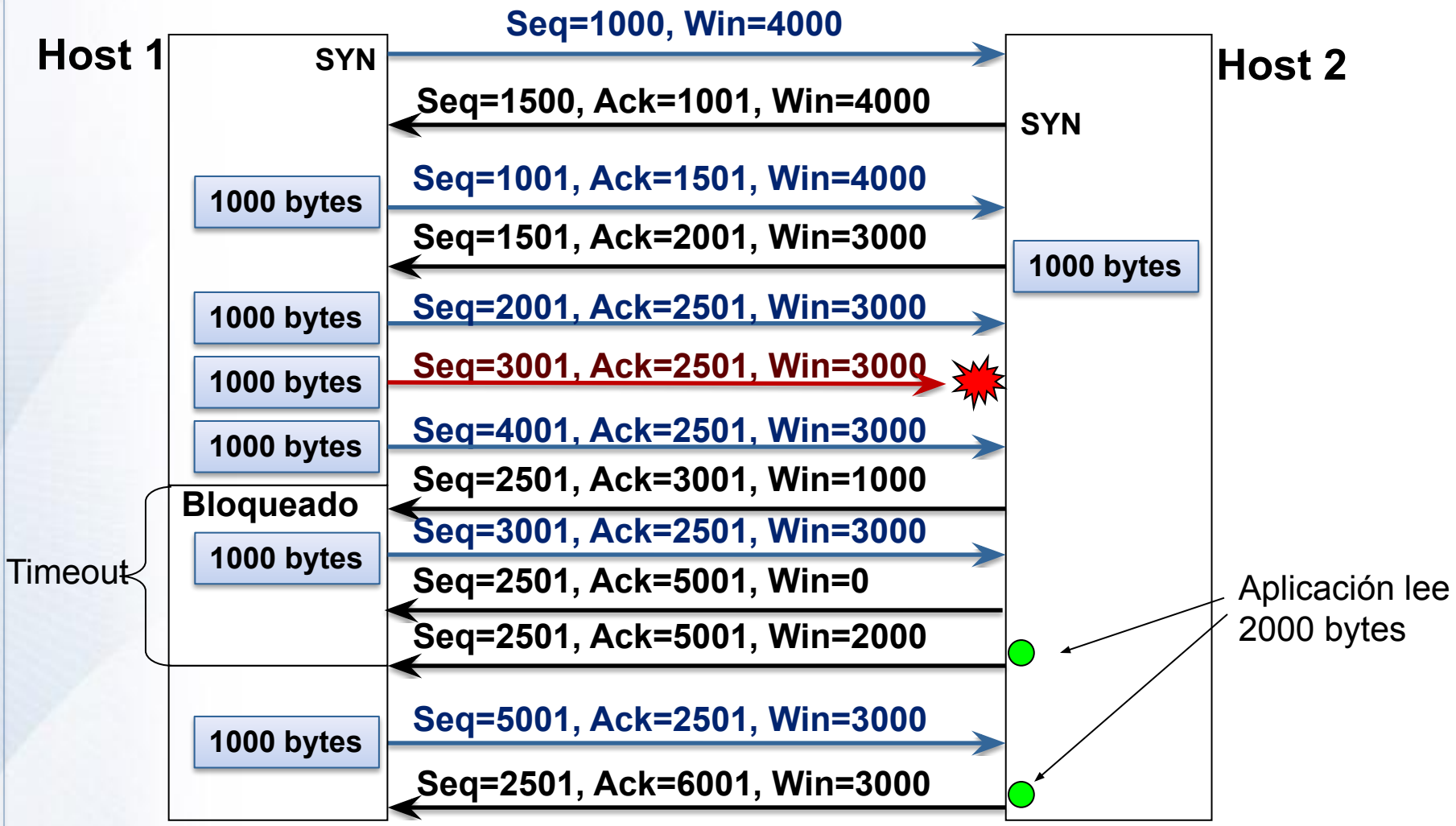
Pérdida de un paquete

Retransmisión con retroceso n



Pérdida de un paquete

Retransmisión con repetición selectiva





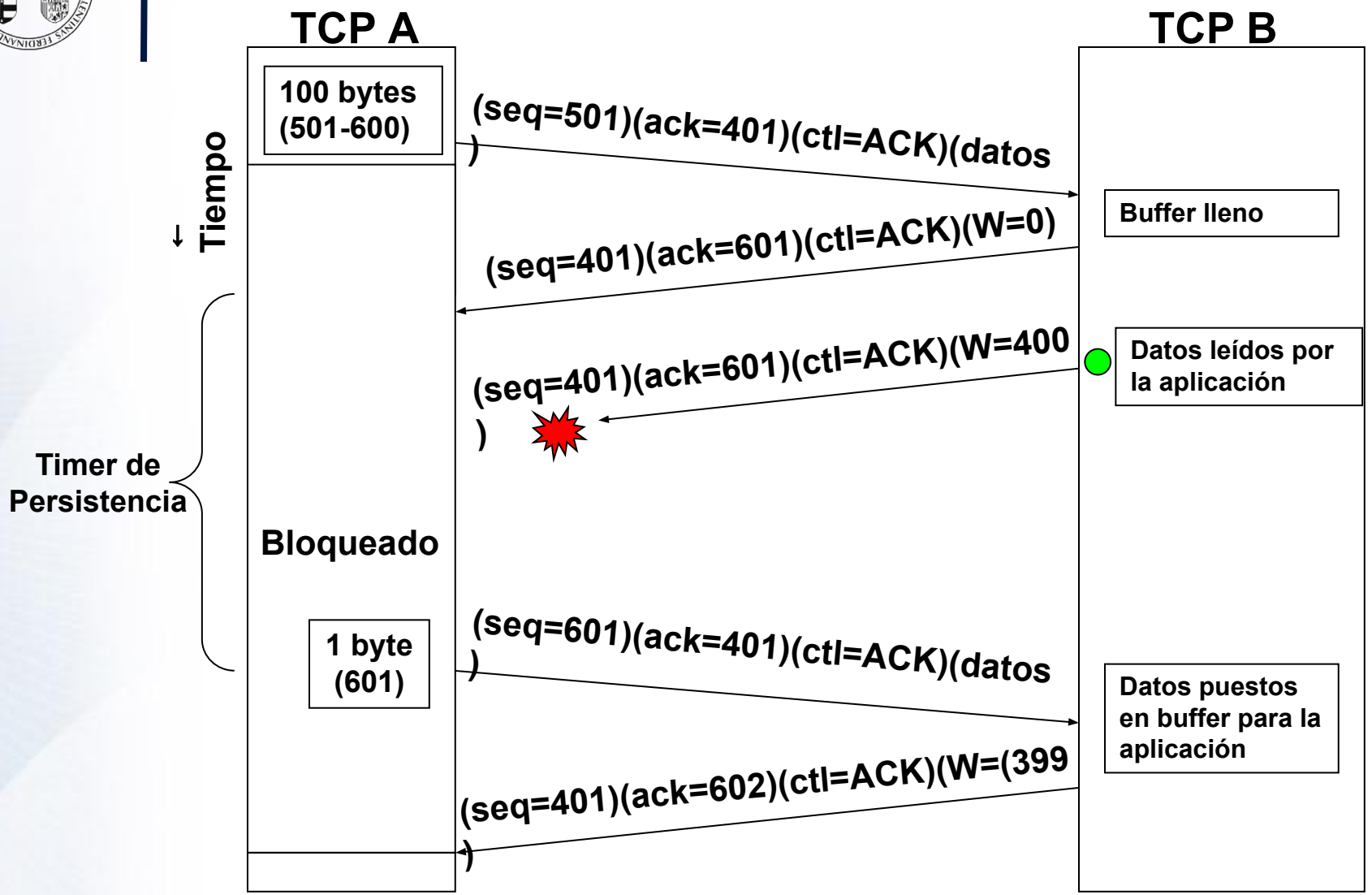
Intercambio de datos
casos excepcionales

- Datos ‘Pushed’ (bit PSH) (Apl TCP)
  - La aplicación pide al TCP emisor que envíe esos datos lo antes posible. El TCP receptor los pondrá a disposición de la aplicación de inmediato, para cuando ésta le pida datos. Ejemplo: telnet.
- Datos Urgentes (bit URG y Urgent Offset)
  - Los datos se quieren entregar a la aplicación remota sin esperar a que esta los pida. Ejemplo: abortar un programa con CTRL-C en una sesión telnet

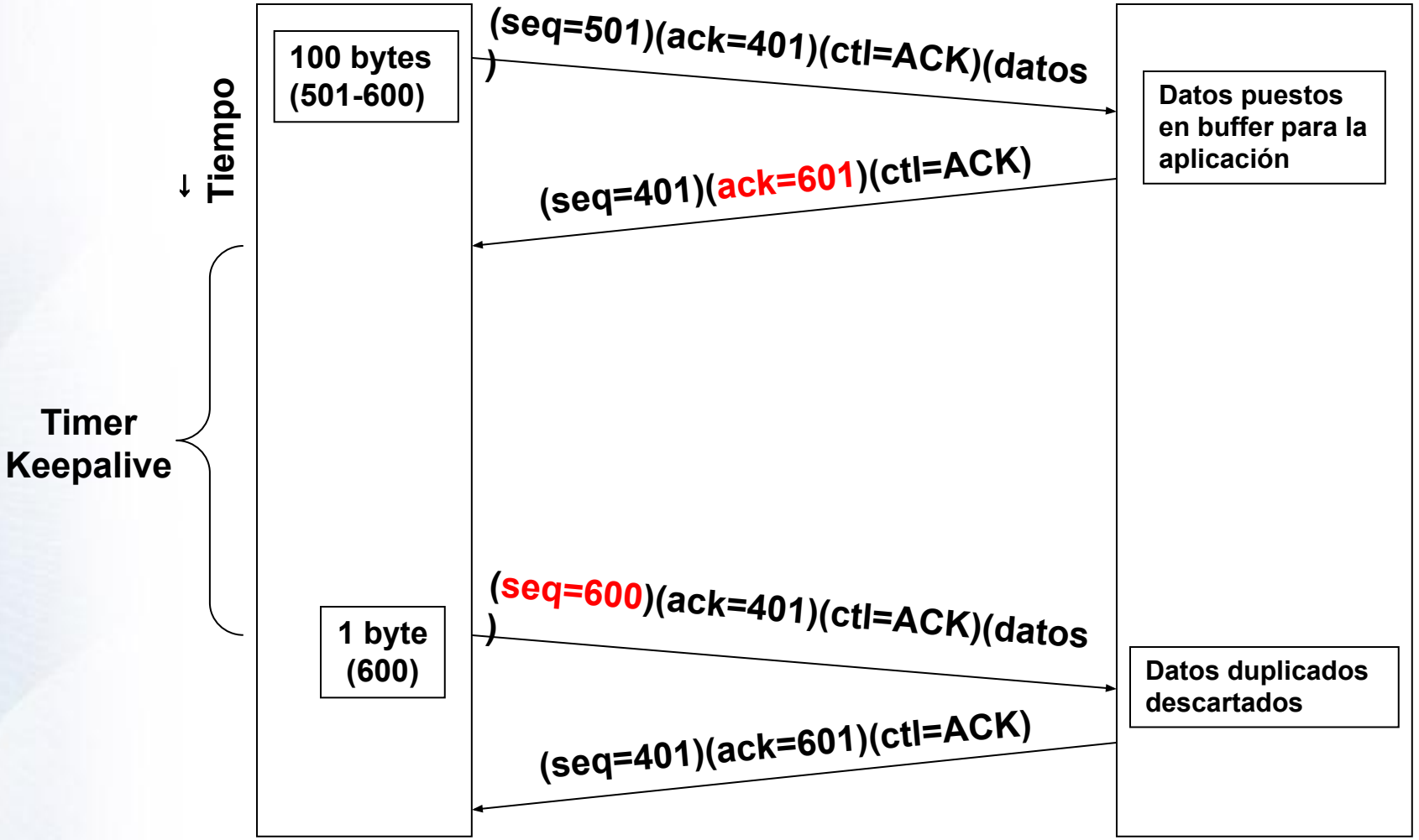
Timer de persistencia

- Mientras la ventana está cerrada el TCP emisor puede enviar de vez en cuando un segmento con un byte de datos; esto provoca el envío de un ACK por parte del receptor y evita el bloqueo que se podría producir en caso de pérdida de un segmento anunciando una ventana mayor que cero
- La frecuencia con que el TCP emisor envía los reintentos se fija en el 'Timer de Persistencia'

Timer de Persistencia



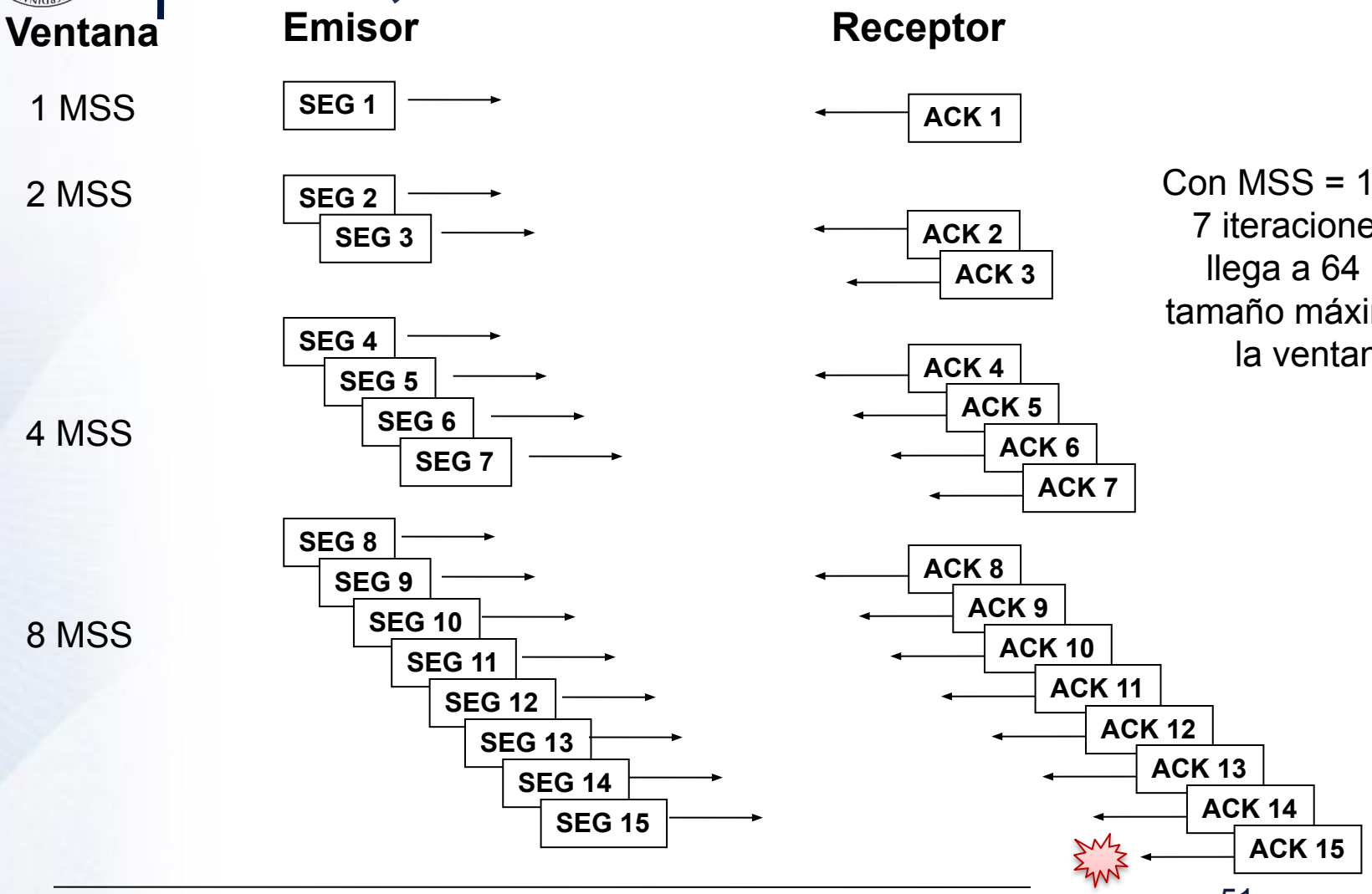
Mensajes de *Keepalive*



Control de congestión en TCP

- Cuando hay congestión TCP ha de reducir el flujo
- El mecanismo para detectarla es implícito, por la pérdida de segmentos. Cuando ocurre TCP baja el ritmo
- Se presupone que la red es altamente fiable a nivel físico y que las pérdidas se deben a congestión únicamente. Cuando no es así (redes con errores) bajar el ritmo es contraproducente
- Además de la ventana de control de flujo (dictada por el receptor y transmitida en la cabecera TCP) el emisor tiene una ventana de control de congestión, que ajusta a partir de los segmentos perdidos. En cada momento se usa la más pequeña de ambas
- El mecanismo de control de congestión de TCP se denomina *slow-start* (arranque lento) y fue diseñado por Van Jacobson en los años 80

Funcionamiento de slow start, fase 1



Con MSS = 1KB en 7 iteraciones se llega a 64 KB, tamaño máximo de la ventana



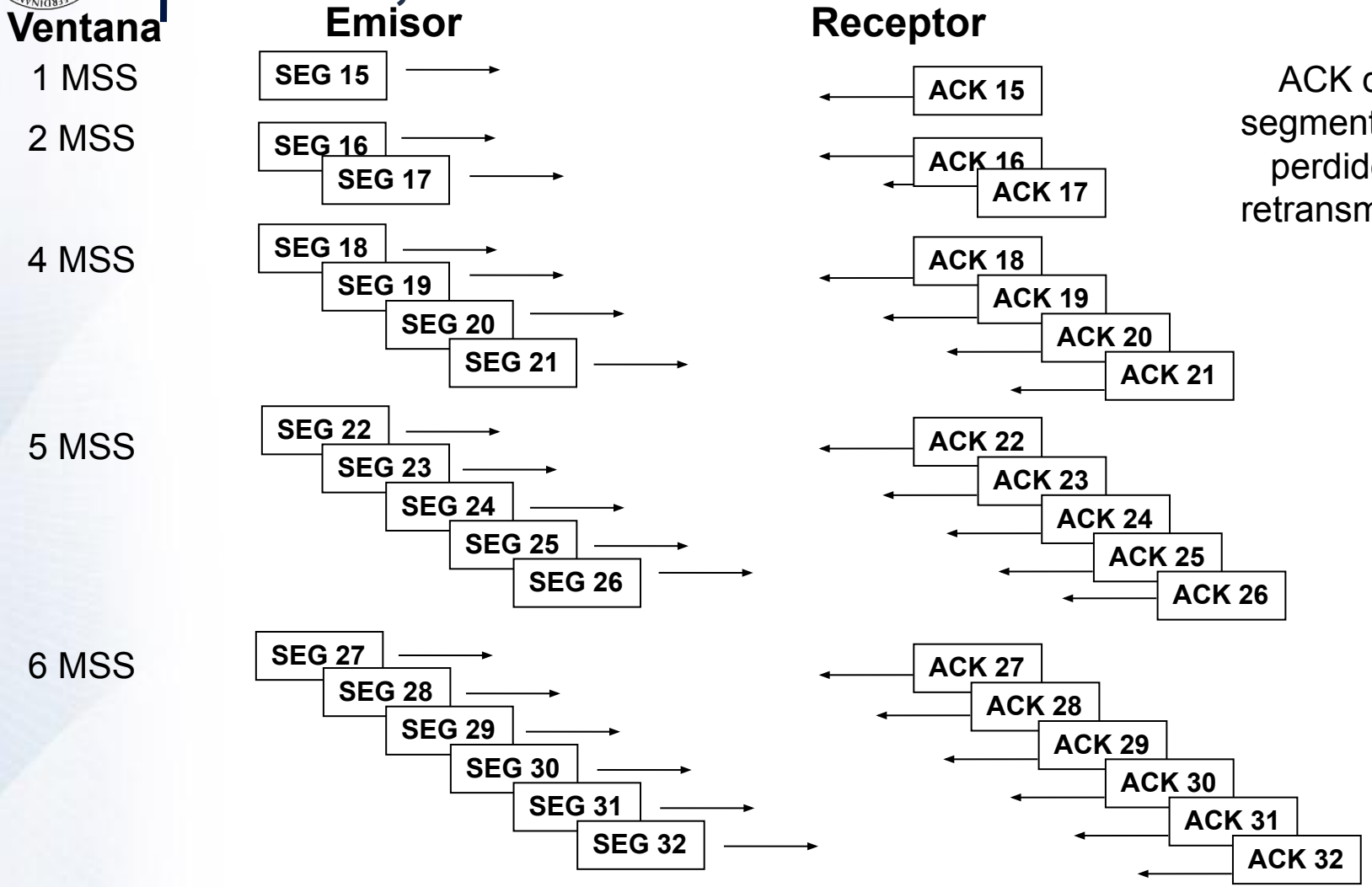
Slow Start (primera fase)

- Inicialmente la ventana de congestión tiene el tamaño de un MSS (Maximum Segment Size)
- Por cada segmento enviado con éxito la ventana se amplía en un MSS
- En la práctica esto supone un crecimiento exponencial (en potencias de dos)
- Si la ventana de congestión supera a la de control de flujo se aplica ésta con lo cual aquella deja de crecer

Slow start (segunda fase)

- Cuando se pierde un segmento:
  - La ventana de congestión vuelve a su valor inicial
  - Se fija un 'umbral de peligro' en un valor igual a la mitad de la ventana que había cuando se produjo la pérdida
  - La ventana de congestión crece como antes hasta el umbral de peligro; a partir de ahí crece en sólo un segmento cada vez

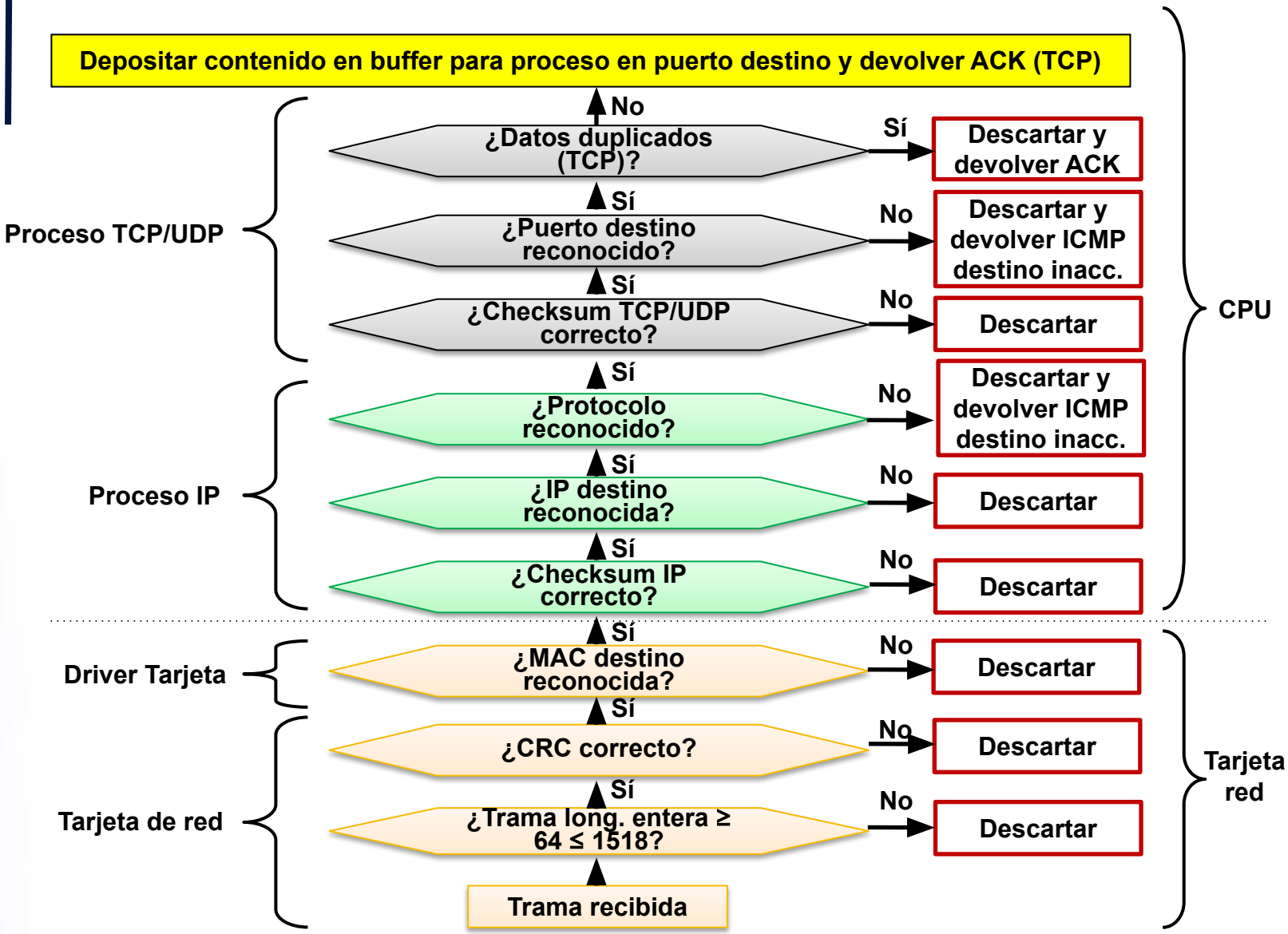
Funcionamiento de slow start, fase 2



ACK del segmento 15 perdido y retransmitido

Timer de retransmisión

- Debe ser adecuado para la comunicación:
  - Si es excesivo se esperará innecesariamente
  - Si es muy corto se harán reenvíos innecesarios
- Como la fluctuación es muy grande se utilizan mecanismos autoadaptativos. A partir de los ACK se mide el tiempo de ida y vuelta o Round Trip Time (RTT)
- La estimación de este timer es crucial para el correcto funcionamiento del 'slow-start'


Depositar contenido en buffer para proceso en puerto destino y devolver ACK (TCP)
Proceso TCP/UDP
¿Datos duplicados (TCP)?
Sí
Descartar y devolver ACK
No
¿Puerto destino reconocido?
Sí
Descartar y devolver ICMP destino inacc.
No
¿Checksum TCP/UDP correcto?
Sí
Descartar
No
Proceso IP
¿Protocolo reconocido?
Sí
Descartar y devolver ICMP destino inacc.
No
¿IP destino reconocida?
Sí
Descartar
No
¿Checksum IP correcto?
Sí
Descartar
No
CPU
Driver Tarjeta
¿MAC destino reconocida?
Sí
Descartar
No
Tarjeta de red
¿CRC correcto?
Sí
Descartar
No
¿Trama long. entera ≥ 64 ≤ 1518?
Sí
Descartar
No
Tarjeta red
Trama recibida